EF234405 Design & Analysis of Algorithms

Classes D, IUP, E, F, G

FINAL EXAM — Group Capstone Project

"Design It, Prove It, Build It, Measure It"

Release Date	5 June 2026
Start Date	10-11 June 2026
Deadline	18 June 2026, at 23:59 WIB
Late Penalty	A penalty of 0.15% of the grade per minute is applied to late submissions.
Exam Type	Open-Book, Group Project (max. 3 students per team). Groups may mix students from Classes D, IUP, E, F, and G in any combination.
Weight	Final Exam — 100 points. Marking scheme: Design 20 · Implementation 50 · Analysis & Evaluation 25 · Conclusion 5.

This Final Exam is a capstone. Unlike Quiz 2 (a build-something project) and the written midterm-style finals (proofs and complexity), this single project asks you to exercise the entire Design & Analysis of Algorithms workflow on one coherent problem: you will DESIGN a solution, PROVE it correct and analyse its complexity, BUILD a working program, and MEASURE its behaviour empirically. Each verb is graded.

1. Submission Guidelines

Email To	MM Irfan Subakti [yifana(at)gmail(dot)com]
CC	Fellyla Fiorenza Wilianto [fellyla(dot)hyuga(at)gmail(dot)com], Putri Meyliya Rachmawati [putrimeyliyaalfath(at)gmail(dot)com], Syalbia Noor Rahmah [syalbiaishere(at)gmail(dot)com], Iffa Amalia Sabrina [iffaamaliasabrina(at)gmail(dot)com], Muhammad Rafi Budi Purnama [mrafibudip(at)gmail(dot)com], Aqila Zahira Naia Puteri Arifin [aqilazhn05(at)gmail(dot)com], Rhenaldy Chandra [itsrhenaldy(at)gmail(dot)com] Ricardo Supriyanto [ricardo(dot)supriyanto08(at)gmail(dot)com], Amelia Nova Safitri [amelianovasafitri(at)gmail(dot)com], Thalyta Vius Pramesti [thalytapramesti(at)gmail(dot)com]
Email Subject	EF234405_DAA_FIN_StudentID1_Name1_StudentID2_Name2
File Submission	A single ZIP archive containing exactly two files: (1) Report.pdf (design, analysis, implementation, evaluation, conclusion + appendices), and (2) Declaration.pdf (the signed Academic Integrity Pledge of all members).
Filename Format	EF234405_DAA_FIN_StudentID1_Name1_StudentID2_Name2.ZIP
GitHub Repository	In the body of your email, include a PUBLIC link to the GitHub repository containing your complete source code. If necessary, commit history is used to assess each member's individual participation, so commit under your own account throughout the week — not in a single final push.

2. The Task

Working in a team of up to three, choose a real-world problem that can be modelled with a data structure from this course (most naturally a weighted graph) and build a working program that solves

it. Your project must satisfy every mandatory requirement below, then be written up in a professional report that follows the section structure in §4.

2.1 Mandatory Requirements (all four are required)

1. **Model.** Formulate a real-world problem as a precise, well-defined computational problem. If you use a graph, state explicitly what the vertices, edges, edge weights, source/targets, objective, and constraints represent.
2. **Two algorithms.** Implement at least TWO algorithms covered in the course and beyond that, solve (or bound) your problem. At least one must be a non-trivial algorithm — for example, Dijkstra, Bellman–Ford, Floyd–Warshall, Prim–Jarník, Kruskal, topological-order DAG shortest/longest path, A*, a dynamic-programming algorithm (LIS, 0/1 knapsack, edit distance, matrix-chain), or an MST-based 2-approximation for metric TSP, etc. The second algorithm may be a simpler baseline (e.g., a brute-force or naïve method) used for comparison.
3. **Comparable.** The two algorithms must run on the SAME problem instances so you can compare them — either they must produce the same answer (which you cross-check), or you must document a clear quality-versus-speed trade-off (e.g., exact vs. approximate).
4. **Working software on GitHub.** Deliver runnable code in a public GitHub repository that a TA can clone and run by following your README, plus a single script that reproduces your benchmark.

2.2 Technical Constraints (so results are meaningful and comparable)

- **Scale.** Your benchmark must exercise the core algorithm on non-trivial inputs — at least $n \geq 1{,}000$ vertices/items (or a clearly justified, domain-appropriate scale).
- **Sweep.** Benchmark at least 5 distinct input sizes spanning at least two orders of magnitude (e.g., 100 → 10,000).
- **Reproducibility.** Provide one command (a script or documented entry point) that regenerates your timing data; fix and report random seeds.
- **Own the core.** You may use a library for supporting structures (heaps, I/O, plotting), but the core algorithmic logic must be YOUR code. Calling a library's shortest-path/MST function as your "implementation" earns 0 for that algorithm.
- **Language.** Any language (C/C++, C#, Java, Python, Rust, Go, JavaScript/TypeScript, …). State the version.
- **Attribution.** Every external snippet, dataset, or library must be cited in the report and the README.

2.3 Example Application Ideas (you may choose your own)

These are starting points only — originality is rewarded. Each names a plausible algorithm pairing:

- **Smart delivery/ride routing —** model a road map as a weighted graph; Dijkstra (binary heap) for fastest route vs. Bellman–Ford or a naïve all-pairs baseline.
- **Maze/dungeon game with pathfinding —** BFS/Dijkstra/A* to chase or escape; compare A* against uninformed search on the same maps.
- **Resilient network design —** build the cheapest connected backbone with Prim–Jarník vs. Kruskal (Union-Find); compare on random vs. clustered topologies.
- **Flight/transit journey planner —** min-hops (BFS) vs. min-cost (Dijkstra), or fewest-transfers vs. cheapest fare.
- **Sightseeing tour planner —** MST-based 2-approximation for metric TSP vs. brute force on small instances to verify the ≤ 2× bound empirically.

- **Dependency/build scheduler —** topological sort + DAG longest path (critical path) vs. a naïve scheduler; detect cycles.

3. Marking Scheme & Point Breakdown (100 pts + up to 10 bonus)

Your project is graded on the same four-category scheme used throughout EF234405 (and in Quiz 2). The TA grade sheet mirrors these categories and criteria exactly, so writing your report in this order makes grading fast and fair for everyone.

Category	Weight	Points
Design	20%	**20**
Implementation	50%	**50**
Analysis & Evaluation	25%	**25**
Conclusion	5%	**5**
Total	**100%**	**100**

The detailed criteria below show how each category is awarded. (Administrative compliance — a valid signed declaration, correct email etiquette, and filename, and on-time submission — is checked separately; see §6.)

Design [20 pts]

Criterion	Pts	Where
(D1) Problem statement & real-world motivation: what problem, why it matters, who would use it.	**4**	Report §1
(D2) Formal model: precise inputs/outputs, objective and constraints; if a graph, define V, E, the weight function w, and source/target semantics.	**5**	Report §1
(D3) Algorithm selection & justification: which two algorithms, why each fits the model, and the trade-off you expect to observe.	**5**	Report §1
(D4) Data structures & system architecture: representations chosen (adjacency list/matrix, priority queue, union-find, …) with justification, plus a module/architecture diagram.	**6**	Report §1

Implementation [50 pts]

Criterion	Pts	Where
(I1) Core algorithm A implemented correctly and from scratch; produces correct results on the included test cases.	**18**	GitHub + §2
(I2) Comparison/baseline algorithm B implemented correctly.	**10**	GitHub + §2
(I3) End-to-end application/demo (CLI, GUI, web, or game) that actually solves the stated problem at the required scale.	**10**	GitHub + §2
(I4) Code quality: modular, readable, sensibly named, documented; no copy-pasted duplication or dead code.	**6**	GitHub
(I5) GitHub & reproducibility: README with build/run/benchmark steps, a one-command benchmark, a tidy repo, and a commit history showing all members contributed.	**6**	GitHub + §4

Analysis & Evaluation [25 pts]

Criterion	Pts	Where
(A1) Correctness: prove or rigorously justify that algorithm A is correct/optimal. For an approximation algorithm, state and justify its approximation guarantee.	**7**	Report §3
(A2) Complexity: derive worst-case time complexity (best/average where relevant), explaining each dominant term; state space complexity.	**6**	Report §3
(A3) Comparative analysis: contrast A vs B asymptotically and identify the input regimes where each is preferable.	**3**	Report §3
(A4) Empirical study: experimental setup (machine, sizes, timing method) plus results — clear tables and at least one runtime-vs-size plot per algorithm.	**5**	Report §3
(A5) Theory vs. practice & cross-check: estimate the empirical growth exponent and compare it to the derived complexity; cross-check that A and B agree where they should.	**4**	Report §3

Conclusion [5 pts]

Criterion	Pts	Where
(C1) Honest summary of findings, limitations, lessons learned, and concrete future work — and the contribution table (each member's % and role).	**5**	Report §4

Bonus [up to +10, total capped at 100]

Awarded only for clearly exceptional work, such as a third, qualitatively different algorithm with comparison (e.g., exact + approximation + heuristic); a rigorous proof of an approximation ratio or lower bound; or an unusually strong empirical study (confidence intervals, multiple input families, profiling).

4. Required Report Structure

Submit one PDF report. Use these exact sections (they map 1-to-1 onto the rubric). Suggested length 12–25 pages, including figures; quality over volume.

5. Title page — project title, all member names + Student IDs, class, date, GitHub link.
6. §1 Design — the Design category (problem, formal model, algorithm choice, data structures + architecture diagram).
7. §2 Implementation — the Implementation category in prose: module overview, key code excerpts (not the whole listing), how to build/run, screenshots of the demo, and a pointer to the GitHub repo + benchmark command.
8. §3 Analysis & Evaluation — the Analysis & Evaluation category (correctness proof/justification, complexity derivation, A-vs-B comparison, experimental setup, tables, plots, and theory-vs-practice with a correctness cross-check).
9. §4 Conclusion — the Conclusion category (findings, limitations, lessons, future work) plus the contribution table.
10. References & Appendix (optional) — citations; extra figures/raw data.

5. Plagiarism & Academic Honesty Policy

To prevent plagiarism and cheating, every student must sign and include the following pledge in Declaration.pdf. The signature may be handwritten or digital. Failure to do so results in a grade of zero (0).

> *"By the name of Allah (God) Almighty, I hereby pledge and declare that I have completed this Final Exam project as part of my team's independent work. I have not engaged in cheating, plagiarism, or received unauthorized assistance in any form. I further declare that any use of AI tools was limited to a supportive role (such as for grammar checking or debugging) and that the final solution is the product of my team's own intellectual effort. I understand that I will accept all consequences if I am found to have violated this academic integrity pledge."*

[City, e.g., Surabaya], [Date, e.g., 18 June 2026]

<Your Signature> [Full Name] [Student ID, e.g., 5025221xxx]	*<Your Signature>* [Full Name] [Student ID, e.g., 5025221xxx]	*<Your Signature>* [Full Name] [Student ID, e.g., 5025221xxx]

Contribution declaration. To ensure fairness, clearly state each member's percentage of contribution and their specific role. Example:

- **Indah — 34%:** Designed the model and architecture; implemented Dijkstra (core).
- **Turangga — 33%:** Implemented the baseline; built the demo; benchmark harness.
- **Muliawati — 33%:** Correctness proof and complexity analysis; evaluation, plots, and final report.

6. How You Will Be Graded (transparency)

A few principles your TAs apply, so there are no surprises:

- **Whole-project view.** Each of the four verbs — design, prove, build, measure — is worth real points. A brilliant program with no analysis, or an elegant proof with no working code, both lose heavily.
- **It must run.** If the TA cannot build/run your project from the README, implementation points are at risk. Test on a clean checkout before submitting.
- **Show your reasoning.** Stated complexities and proofs must be justified, not merely asserted.
- **Individual fairness.** The group earns the project grade; commit history and the contribution table may adjust an individual's grade up or down for unequal participation.
- **Penalties & floor.** Late submissions lose 0.15% per minute. BC (61) is the default minimum grade. A valid signed Declaration.pdf and correct email/filename etiquette are required compliance items; confirmed plagiarism is handled separately by the instructor and overrides the BC floor.

How scores become letter grades

Your numeric score converts to an ITS letter grade as follows. Importantly, BC (61) is the default minimum grade — every enrolled student starts from BC, and the project raises the grade from there — so in a healthy cohort, the grades cluster in the BC–A range.

Grade	Score	What it looks like
A	86–100	Clean, working software solving a real problem at the required scale; two correct algorithms; a sound correctness proof and correct complexity derivation; experiments whose measured growth matches theory with a thoughtful comparison; reproducible repo with balanced contributions.
AB	76–85	Strong, working software and both algorithms; correctness proof and complexity mostly right with only minor gaps; benchmarks present and reasonably connected to theory.
B	66–75	Working software and both algorithms; analysis partly developed, but with real gaps; benchmarks thin or only loosely tied to theory.
BC	61–65	Default/baseline grade. Something runs, and at least one algorithm works; analysis superficial or partly wrong; little or no empirical work, and the band a non-submission lands in, since BC is the floor.

The full ITS scale also defines C (56–60), D (41–55), and E (0–40); because of the BC floor, these do not occur in normal grading, and E is reserved for confirmed academic dishonesty handled by the instructor.

7. Tips & FAQ

- **Start with the model.** A crisp formal model in §1 makes the proof, the code, and the experiments fall into place.
- **Pick algorithms that contrast.** Dijkstra vs Bellman–Ford, Prim vs Kruskal, A* vs BFS, or exact vs approximation give the richest comparison and easy points in criteria A3 and A5.
- **Automate timing.** A loop over sizes that writes a CSV, plus a tiny plotting script, is enough — and it makes the Analysis & Evaluation experiments reproducible.
- **“Can two teammates be in different classes?”** Yes — any mix of D, IUP, E, F, G.
- **“Can we reuse our Quiz 2 idea?”** You may build on the same domain, but the Final adds the mandatory second algorithm, the correctness proof, and the empirical complexity study; resubmitting Quiz 2 unchanged will not pass.
- **Questions?** Reach out to any TA listed above. Good luck, and have fun building something you are proud of!

Appendix A — A Complete Worked Specimen (for guidance only)

Read this, do not copy it. This specimen shows exactly what an A-grade submission looks like across all four categories, so you know the standard and the structure expected of you. It deliberately solves a DIFFERENT problem (minimum-cost network design) from anything you should hand in. Submitting this project—or a thin variation of it—is plagiarism and will be treated under the policy in §5. Use it as a map for depth and rigour, then build your own original project.

Specimen project: "GridGuard" — designing the cheapest resilient fibre backbone for a campus.

A.1 Design [20]

Problem & motivation (D1). A campus must lay fibre that connects every building, digging as few trenches as possible while keeping the whole campus connected. Facilities planners face this whenever they expand a network; cheaper connected layouts free budget for redundancy elsewhere.

Formal model (D2). Model the campus as a weighted, undirected graph G = (V, E): V = buildings; E = feasible trench routes between buildings; w(e) ≥ 0 = the cost of route e. We seek a spanning tree T ⊆ E (a connected, acyclic subgraph touching every building) of minimum total weight — a Minimum Spanning Tree (MST). Input: G. Output: the edge set of an MST and its total cost.

Algorithm selection (D3). Algorithm A — Kruskal: sort the edges and repeatedly add the cheapest edge that does not create a cycle, using Union-Find. Algorithm B — Prim (binary heap): grow one tree from a seed, building, always adding the cheapest edge leaving it. Both compute an MST, so they must report the SAME total cost (our correctness cross-check); we expect Kruskal to shine when edges are few or pre-sorted, and heap-Prim to stay competitive as the graph gets dense.

Data structures & architecture (D4). Kruskal uses an edge list plus a Union-Find (disjoint-set) structure with path compression and union by rank; Prim uses an adjacency list plus a binary min-heap and a visited[] array. Modules: GraphGenerator → MST interface implemented by Kruskal and Prim → Visualizer/Demo → BenchmarkHarness (writes CSV) → plotting script. A one-page block diagram accompanies this in the report.

What earns the marks: a concrete problem with a real user; V, E, and w defined precisely; two contrasting algorithms with a predicted trade-off; justified data structures and an architecture diagram.

A.2 Implementation [50]

Algorithm A — Kruskal with Union-Find (I1):

```
KRUSKAL(G, w):
  sort edges E by weight ascending
  for each vertex v:  parent[v] = v;  rank[v] = 0
  T = {}                              // chosen MST edges
  for each edge (u, v) in sorted E:
    if FIND(u) != FIND(v):            // adding it joins two trees
      add (u, v) to T;  UNION(u, v)
  return T

FIND(x):  if parent[x] != x: parent[x] = FIND(parent[x])   // path compression
          return parent[x]
UNION(a, b):  ra = FIND(a); rb = FIND(b)                   // union by rank
          if rank[ra] < rank[rb]: swap(ra, rb)
          parent[rb] = ra;  if rank[ra]==rank[rb]: rank[ra] += 1
```

Algorithm B — Prim with a binary min-heap (I2):

```
PRIM(G, w, seed):
  for each v: key[v] = +infinity; inTree[v] = false; parent[v] = NIL
  key[seed] = 0;  PQ = min-heap of (key, vertex);  PQ.push((0, seed))
  T = {}
  while PQ not empty:
    (k, u) = PQ.pop()
    if inTree[u]: continue            // stale entry, skip
    inTree[u] = true
    if parent[u] != NIL: add (parent[u], u) to T
    for each edge (u, v) with weight w(u,v):
      if not inTree[v] and w(u,v) < key[v]:
        key[v] = w(u,v); parent[v] = u; PQ.push((key[v], v))
  return T
```

Demo (I3) & code quality (I4) & repo (I5). The demo draws the campus graph and highlights the chosen MST, printing its total cost; it runs on generated maps of 1,000+ buildings. The two algorithms sit behind one MST interface (no duplicated logic), with brief docstrings and meaningful names. The public repo has /src, /bench (harness + CSV), /docs, and a README whose single command reproduces the benchmark; commit history shows all members.

What earns the marks: both algorithms are the team's own code (a heap or sort library is fine); the demo genuinely solves the campus problem at scale; the repo builds and benchmarks from the README with all members committing.

A.3 Analysis & Evaluation [25]

Correctness (A1) — the cut property. Theorem: for any partition of V into (S, V\S), if e is a minimum-weight edge crossing the cut, then some MST contains e.

Proof. Let T be any MST. If $e \in T$, we are done, so suppose $e = (u, v) \notin T$ with $u \in S$, $v \notin S$. Adding e to the spanning tree T creates exactly one cycle; that cycle starts in S and returns to S, so it crosses the cut at least one OTHER edge e'. Because e is a minimum-weight edge crossing the cut, $w(e) \le w(e')$. Form $T' = T - e' + e$: removing e' from the cycle and inserting e keeps the graph a spanning tree, and $w(T') = w(T) - w(e') + w(e) \le w(T)$. Since T is minimum, $w(T') = w(T)$, so T' is also an MST — and it contains e. ■

Both algorithms only ever add safe (minimum crossing) edges: Kruskal's next accepted edge is the globally lightest edge joining two different components, i.e. the lightest edge crossing the cut between one component and the rest; Prim's next edge is the lightest edge leaving the current tree S, i.e. the lightest edge crossing (S, V\S). By the cut property each addition is safe, so by induction both build an MST.

What earns the marks: the cut property is stated and proved (cycle-and-exchange argument), and each algorithm is shown to add only safe edges — not merely asserted to 'work'.

Complexity (A2). Kruskal: sorting E edges costs $O(E \log E) = O(E \log V)$ (since $E \le V^2$); the E Union-Find operations cost almost $O(E)$ with path compression + union by rank (inverse-Ackermann α). So Kruskal is $O(E \log V)$. Prim with a binary heap: each vertex is popped once (V pops), and each edge can push once (E pushes), every heap op $O(\log V) \rightarrow O((V + E) \log V)$. Space for both is $O(V + E)$.

Algorithm	Time (worst case)	Space
Kruskal	$O(E \log V)$	$O(V + E)$
Prim (binary heap)	$O((V + E) \log V)$	$O(V + E)$

Comparison (A3). On sparse campuses (E = O(V)), both are near O(V log V); Kruskal is especially fast when edges are few or already sorted, while heap-Prim avoids a global sort and stays competitive as the graph becomes dense. Both crush a naïve O(V·E) check.

Empirical study (A4) — illustrative reference values (one laptop, 5-run average, random graphs E ≈ 5V, fixed seed):

V	E ≈ 5V	Kruskal (ms)	Prim (ms)	Same cost?
100	500	0.2	0.2	yes
500	2,500	1.1	1.0	yes
1,000	5,000	2.4	2.2	yes
5,000	25,000	14	13	yes
10,000	50,000	30	28	yes

Theory vs practice & cross-check (A5). On log–log axes, both series are nearly straight with slope ≈ 1.1, consistent with the near-linear O(V log V) behaviour on sparse graphs (the log factor only gently bends the line). Crucially, both algorithms returned the IDENTICAL total cost on every instance (the 'Same cost?' column), confirming the implementations against each other. Small deviations at tiny V come from fixed overheads.

What earns the marks: time AND space derived with each term explained; a regime-based comparison; ≥ 5 sizes over two orders of magnitude with a plot; the empirical exponent tied back to the theory; and an A-vs-B correctness cross-check.

A.4 Conclusion [5]

Both Kruskal and Prim deliver an optimal backbone; on our sparse campus graphs, they perform almost identically, with Kruskal marginally ahead thanks to a cheap sort and fast Union-Find. Limitations: static costs and a single seed for Prim. Future work: support live cost updates and compare against an approximation for the budget-constrained variant. Contribution table example — Member 1 (34%): model + Kruskal + proof; Member 2 (33%): Prim + demo + benchmark harness; Member 3 (33%): complexity analysis, evaluation, plots, report.

What earns the marks: honest findings and limitations, concrete future work, and a clear contribution table.

That is the full arc — Design, Implementation, Analysis & Evaluation, Conclusion — at the A standard. Match this depth on your own original problem, and you will do very well.